

CUI Portal Application





COMSATS University, Islamabad Pakistan

Project Technical Specification Document

CUI Portal Application

Submitted By:

Ali Haider	FA23-BSE-014
Talal Hamid	FA23-BSE-108
Aimen Shahzad	FA23-BSE-111

Supervisor

Mr. Muzaffar Iqbal

Submission Date: (03-01-2025)

Original Version 1.0

Bachelor of Software Engineering (2023-2027)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

Contents

1. Overview of the Application	5
2. System Architecture.....	5
2.1 High-Level Architecture	5
2.2 Major Components	5
2.3 Key Modules and Their Responsibilities.....	6
2.4 Interactions Between Components.....	6
2.5 Tools and Technologies	6
3. Functional Specifications	6
3.1 Core Functionalities	6
3.2 User Interfaces.....	7
3.3 Workflow	7
4. Non-Functional Specifications	7
4.1 Performance Requirements.....	7
4.2 Usability	8
4.3 Security	8
4.4 Scalability	8
4.5 Reliability.....	8
4.6 Portability	8
4.7 Maintainability	8
4.8 Look and Feel.....	8
5. Data Design.....	9
5.1 Data Storage	9
5.2 Database Structure	9
5.3 Entity Descriptions	9
5.4 Data Flow.....	9
5.5 Example Data Representation	10
6. User Interface Design.....	10
6.1 Overview.....	10
6.2 Screens and Features	10
6.3 Design Consistency	11

6.4 Accessibility.....	12
7. Integration Points.....	12
7.1 Overview.....	12
7.2 External Libraries.....	12
7.3 Internal Modules	12
7.4 Interactions Between Modules	13
7.5 Potential Integration Extensions.....	13
8.Core OOP Concepts	13
8.1.Advanced OOP Concepts	14
8.2.GUI and Event Handling	14
8.3.Exception Handling	14
8.4..Java Programming Constructs	14
8.5.Collections and Data Structures	14
8.6.File and Data Handling.....	15
8.7.Java Keywords and Features.....	15
9. User Interface (UI) Design.....	15
9.1. Overall Design Philosophy	15
9.2. Mockups or Wireframes	18
9.3. Explanation of Key Screens	18
9.4. Interaction Flow	20
Class Diagram.....	21

Technical Specification Document

CUI Portal

1. Overview of the Application

- **Purpose:**
The provided application, **CUI Portal Application**, is designed to manage various academic functionalities, including user authentication, quiz management, profile creation, and attendance tracking.
- **Scope:**
The application provides an interactive graphical user interface (GUI) built using **Java Swing**, integrating themes for enhanced user experience. It features multiple modules such as:
 - **LogInScreen:** Handles user authentication.
 - **AdminMenuScreen:** Allows administrators to manage students and teachers, view quiz marks, and change themes.
 - **CreateQuizScreen:** Facilitates creating quizzes with multiple-choice questions.
 - **AttemptQuizScreen:** Enables students to attempt quizzes and record their scores.
 - **AttendanceScreen:** Allows teachers to mark student attendance.
- **Core Libraries Used:**
 - **FlatLaf:** For modern look-and-feel themes.
 - **Java Swing:** For GUI components.
- **Target Audience:**
Educational institutions managing academic activities for students and teachers.

2. System Architecture

2.1 High-Level Architecture

The **CUI Portal Application** follows a modular architecture with well-defined components for user management, quiz handling, and attendance tracking. It utilizes **Java Swing** for the GUI, and the application is structured into separate packages for better maintainability and scalability.

2.2 Major Components

1. **Frontend (GUI):**
 - Developed using **Java Swing**.
 - Provides an intuitive interface with various screens, such as login, admin menu, quiz creation, and attendance marking.
 - Incorporates **FlatLaf** themes for a modern look-and-feel.
2. **Backend Logic:**
 - Encapsulated in Java classes within the codes package.

- Handles data operations like quiz generation, user role-specific actions, and attendance marking.
- 3. **Persistence Layer (File Handling):**
 - Data storage is managed via Java object serialization and file handling, enabling the application to read/write user profiles, quizzes, and attendance records.

2.3 Key Modules and Their Responsibilities

- **LogInScreen:**
 - Authenticates users and directs them to role-specific menus (Admin, Teacher, or Student).
- **AdminMenuScreen:**
 - Allows administrators to create and view profiles, delete users, and manage application themes.
- **CreateQuizScreen:**
 - Facilitates quiz creation by enabling administrators or teachers to input questions and correct answers.
- **AttemptQuizScreen:**
 - Displays quizzes to students, captures answers, and calculates scores.
- **AttendanceScreen:**
 - Enables teachers to mark attendance for students in different semesters.

2.4 Interactions Between Components

- GUI components invoke backend methods to perform actions (e.g., saving a quiz or marking attendance).
- Backend logic interacts with the persistence layer to fetch or update data files.

2.5 Tools and Technologies

- **Programming Language:** Java
- **Libraries and Frameworks:** FlatLaf, Java Swing
- **Development Environment:** NetBeans IDE (noted from the comments in the code).

3. Functional Specifications

3.1 Core Functionalities

The **CUI Portal Application** provides a range of functionalities tailored to its users, categorized into administrative, student, and teacher-specific actions:

1. **User Authentication**
 - **LogInScreen** validates user credentials and navigates to the appropriate user dashboard based on their role.
2. **Admin Features**
 - **Profile Management:**
 - Create, view, and delete profiles for students and teachers.
 - **Theme Management:**
 - Change the application theme dynamically using options provided by **FlatLaf**.

- **Quiz Marks:**
 - View student quiz marks.
- 3. **Teacher Features**
 - **Quiz Creation:**
 - Add new quizzes with multiple-choice questions and designate correct answers.
 - **Attendance Management:**
 - Mark student attendance for specific subjects and topics.
- 4. **Student Features**
 - **Quiz Attempt:**
 - Select a subject, attempt quizzes, and receive scores.
 - **Attendance Status:**
 - View attendance records (not explicitly implemented in the provided code but inferred from functionality).

3.2 User Interfaces

- **Login Screen:**
 - Input fields for username and password with role-based redirection.
- **Admin Dashboard:**
 - Buttons for profile creation, theme changes, and quiz mark views.
- **Teacher Dashboard:**
 - Options to manage attendance and quizzes.
- **Student Dashboard:**
 - Modules for attempting quizzes and viewing results.

3.3 Workflow

1. **Login Process:**
 - Users enter credentials and are redirected based on their role.
2. **Admin Actions:**
 - Create or manage profiles and access other admin features.
3. **Teacher Actions:**
 - Generate quizzes and manage attendance for students.
4. **Student Actions:**
 - Attempt quizzes and receive feedback on performance.

4. Non-Functional Specifications

4.1 Performance Requirements

- The application should load and render all GUI components within **2 seconds** for typical user interactions.
- Quiz data retrieval and attendance marking operations should complete within **1 second** under normal conditions.

4.2 Usability

- The user interface should be intuitive and easy to navigate, ensuring smooth interaction for administrators, teachers, and students.
- Consistent themes and button placements across all screens for familiarity.

4.3 Security

- Ensure that user authentication credentials are validated securely to prevent unauthorized access.
- Restrict access to administrative and teacher functionalities based on role-specific permissions.

4.4 Scalability

- The system can accommodate an increasing number of students, teachers, and quizzes by managing data efficiently through serialized files.
- Modular architecture supports adding new features like additional roles or functionalities without significant restructuring.

4.5 Reliability

- The application should maintain consistent operation without unexpected crashes or data corruption.
- All actions, such as creating quizzes or marking attendance, must confirm successful completion to the user.

4.6 Portability

- The application runs on any system with a Java Runtime Environment (JRE), making it platform-independent.

4.7 Maintainability

- Well-structured packages and modular code design ensure easy debugging, updates, and extensions.
- Usage of comments throughout the codebase for better readability and maintainability by other developers.

4.8 Look and Feel

- Integrated with **FlatLaf** themes to offer modern, customizable aesthetics.
- Options for switching between light and dark modes based on user preference.

5. Data Design

5.1 Data Storage

The application uses **Java Object Serialization** for storing and retrieving data. All data related to users, quizzes, and attendance is saved in files, ensuring persistence between sessions.

5.2 Database Structure

Although the application does not use a relational database, its design can be interpreted as structured data files:

- **Users:**
Contains serialized objects representing students, teachers, and administrators.
- **Quizzes:**
Stores quiz questions, options, correct answers, and associated subjects.
- **Attendance Records:**
Tracks attendance for students, including topics and dates.

5.3 Entity Descriptions

1. **User Entities:**
 - **Admin:**
Fields: username, password, role
 - **Teacher:**
Fields: name, ID, subjects[], attendanceRecords[]
 - **Student:**
Fields: name, ID, semester, subjects[], quizScores[], attendancePercentage
2. **Quiz Entity:**
 - Fields: question, options[], correctAnswer, subject
 - Relationships:
 - Associated with a **teacher** for creation.
 - Linked to a **student** for attempts.
3. **Attendance Entity:**
 - Fields: studentID, date, topic, status (Present/Absent)

5.4 Data Flow

- **User Input:**
Data entered by users in the GUI (e.g., quiz answers or attendance marking) is validated and saved to the appropriate file.
- **File Interaction:**
 - Read: Retrieve objects (e.g., student profiles or quizzes) when needed.
 - Write: Update or append data (e.g., new quiz results or attendance records).

5.5 Example Data Representation

1. Student Object (Serialized):

```
java
Copy code
{
  "name": "John Doe",
  "ID": "ST123",
  "semester": "5",
  "subjects": ["Math", "Science"],
  "quizScores": [{"subject": "Math", "score": 8}],
  "attendancePercentage": 85
}
```

2. Quiz Object (Serialized):

```
java
Copy code
{
  "question": "What is 2+2?",
  "options": ["1", "2", "3", "4"],
  "correctAnswer": "4",
  "subject": "Math"
}
```

6. User Interface Design

6.1 Overview

The application features a graphical user interface (GUI) built using **Java Swing**, designed to provide an intuitive and user-friendly experience. Each interface aligns with the role-based functionalities of the application, such as administration, teaching, and learning.

6.2 Screens and Features

1. Login Screen

- **Purpose:** Authenticate users and redirect them to their respective dashboards.
- **Components:**
 - Input fields for **username** and **password**.
 - Role selection (Admin, Teacher, or Student).
 - Login button and error message display for invalid credentials.

2. Admin Dashboard

- **Purpose:** Provide administrators access to core management features.
- **Components:**
 - Buttons for:
 - Creating and viewing **student/teacher profiles**.
 - Deleting users.

- Viewing **quiz marks**.
 - Changing the **application theme**.
 - A centered label indicating the **Admin Dashboard**.
3. **Teacher Dashboard**
- **Purpose:** Enable teachers to manage quizzes and attendance.
 - **Components:**
 - Buttons for:
 - Creating quizzes.
 - Viewing and managing attendance records.
 - Navigating to the main menu.
4. **Student Dashboard**
- **Purpose:** Allow students to interact with quizzes and track attendance.
 - **Components:**
 - Options for:
 - Selecting a subject and attempting quizzes.
 - Viewing results after completing quizzes.
 - Navigating to the main menu.
5. **Quiz Creation Screen**
- **Purpose:** Enable teachers to design quizzes.
 - **Components:**
 - Text fields for the question and answer options.
 - Dropdown to select the correct answer.
 - Button to save or reset the quiz.
 - Label to indicate the quiz subject.
6. **Quiz Attempt Screen**
- **Purpose:** Allow students to attempt quizzes and submit responses.
 - **Components:**
 - Dropdown to select a subject.
 - Display area for questions and multiple-choice options.
 - Buttons for navigating between questions and submitting answers.
 - Score display after quiz completion.
7. **Attendance Screen**
- **Purpose:** Facilitate marking student attendance.
 - **Components:**
 - Dropdown for selecting a semester.
 - Input field for topic name.
 - Display area for student names.
 - Radio buttons for marking **Present** or **Absent**.
 - Buttons to navigate to the next student, reset, or submit attendance.

6.3 Design Consistency

- **Theming:**
 - Themes are managed using **FlatLaf** libraries, supporting light and dark modes.
- **Font Styles:**
 - Consistent use of fonts like **Times New Roman** for labels and buttons.

6.4 Accessibility

- **Keyboard Navigation:**
Buttons and input fields are accessible via keyboard shortcuts where applicable.
- **Error Feedback:**
Clear error messages are displayed for invalid actions (e.g., failed login attempts or missing input).

7. Integration Points

7.1 Overview

The application integrates multiple components and external libraries to provide functionality such as theming, file handling, and user management. These integration points ensure seamless interaction between modules and external dependencies.

7.2 External Libraries

1. FlatLaf

- **Purpose:** Provides modern look-and-feel themes for the GUI.
- **Usage:**
 - Different themes (e.g., FlatDarculaLaf, FlatLightLaf) are applied dynamically based on user preference.
 - Example Integration:

```
java
Copy code
FlatHiberbeeDarkIJTheme.setup();
```

2. Java Swing

- **Purpose:** Core library for creating GUI components.
- **Usage:**
 - Components like buttons, labels, text fields, and frames for building user interfaces.
 - Layout managers for organizing UI elements.

7.3 Internal Modules

1. File Handling Module

- **Purpose:** Manages data persistence through object serialization.
- **Integration Points:**
 - Save and retrieve user profiles, quiz data, and attendance records.
 - Example:

```
java
Copy code
FileHandling.writeObject(studentObject);
```

2. Role-Based Modules

- **Admin Menu:**
Integrates with profile creation and quiz management modules.
- **Student Module:**
Accesses quiz and attendance data for personalized views.
- **Teacher Module:**
Manages quiz creation and attendance marking functionalities.

7.4 Interactions Between Modules

- **Authentication to Dashboards:**
 - User roles are determined during login, redirecting to the appropriate dashboard.
- **Quiz Data Flow:**
 - Teachers create quizzes → Data stored via file handling → Students access and attempt quizzes.
- **Attendance Data Flow:**
 - Teachers mark attendance → Records updated via file handling → Data retrieved for reporting.

7.5 Potential Integration Extensions

- **Database Integration:**
 - Replace file handling with a relational or NoSQL database for enhanced scalability.
- **API Integration:**
 - Introduce RESTful APIs for remote access and third-party integrations (e.g., student portals or learning management systems).

8.Core OOP Concepts

1. **Encapsulation** - Hiding data using private variables and providing public methods for access.
2. **Inheritance** - Extending classes to reuse and add functionality.
3. **Polymorphism**
 - **Compile-Time Polymorphism** (Method Overloading).
 - **Run-Time Polymorphism** (Method Overriding).
4. **Abstraction** - Abstract methods and classes, focusing on what an object does rather than how.
5. **Class and Object** - Definitions and instantiations.
6. **Constructors** - Initializing objects with constructors.

8.1.Advanced OOP Concepts

7. **Method Overriding** - Redefining a method in a subclass.
8. **Method Overloading** - Same method name with different parameters.
9. **Access Modifiers**
 - private
 - protected
 - public
 - Default (package-private).
10. **Static Methods and Variables** - Shared class-level members.
11. **Final Keyword Usage**
 - Final variables (constants).
 - Final methods (prevent overriding).
12. **Abstract Classes and Methods** - Providing base functionality while enforcing implementation in subclasses.
13. **Interfaces** - For multiple inheritance and abstraction.

8.2.GUI and Event Handling

14. **Swing Components** (JButton, JLabel, etc.).
15. **Event Handling** (e.g., ActionListener).
16. **Java Swing Layouts** (GroupLayout, GridBagLayout, etc.).

8.3.Exception Handling

17. **try-catch Blocks** - Handling exceptions.
18. **Throwing Exceptions** (throw keyword).
19. **Checked and Unchecked Exceptions**.

8.4..Java Programming Constructs

20. **Packages** - Organizing code into package structures.
21. **Imports** - Importing external classes and packages.
22. **Composition** (Has-A Relationship) - Objects containing other objects.
23. **Annotations** (e.g., @Override).
24. **Inner Classes** - Classes defined within another class.
25. **Nested Classes** - Static inner classes.

8.5.Collections and Data Structures

27. **ArrayList** - Managing dynamic collections of objects.
28. **Generic Types** - Ensuring type safety in collections.

8.6.File and Data Handling

- 29. **File Handling** - Reading and writing files.
- 30. **Serialization** - Saving and restoring objects.

8.7.Java Keywords and Features

- 31. **this Keyword** - Referring to the current instance.
- 32. **super Keyword** - Accessing parent class methods and constructors.
- 33. **Instance Variables and Methods** - Non-static members.

9. User Interface (UI) Design

9.1. Overall Design Philosophy

- The UI design aims to be clean, intuitive, and user-friendly, with a focus on functionality and ease of navigation.
- The dark theme with contrasting button colors enhances readability and provides a professional aesthetic.
- The design emphasizes consistency across all user roles (Admin, Teacher, Student) to ensure a seamless user experience.

91.1. Color Palette

- **Primary:** #333333 (Dark Gray)
- **Secondary:** #FFFFFF (White)
- **Accent:** #FF0000 (Red)
- **Button Hover:** #666666 (Darker Gray)

9.1.2 Typography

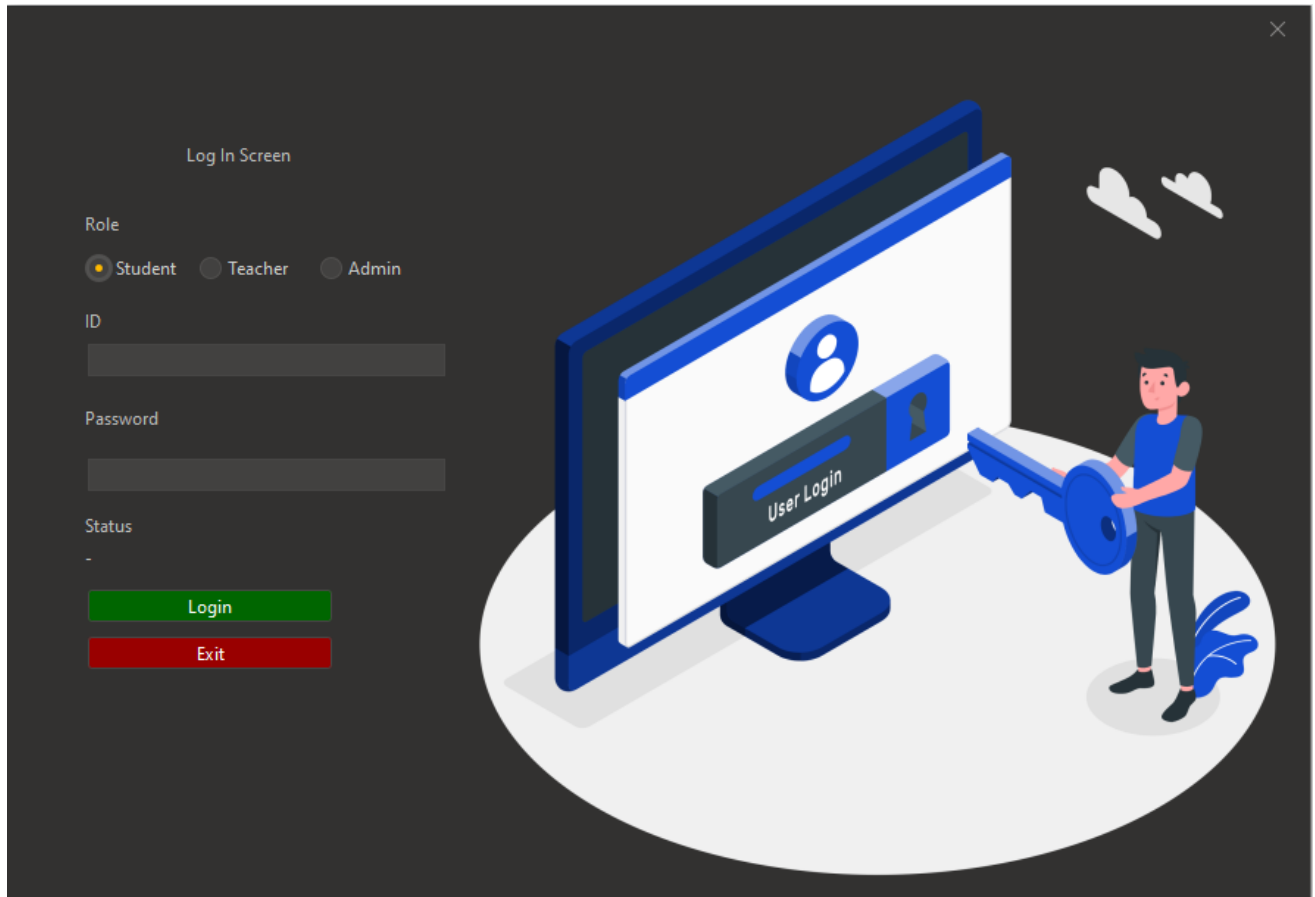
- **Font:** Arial, sans-serif
- **Font Size:**
 - Headings: 16px
 - Body Text: 14px

9.1.3. Iconography

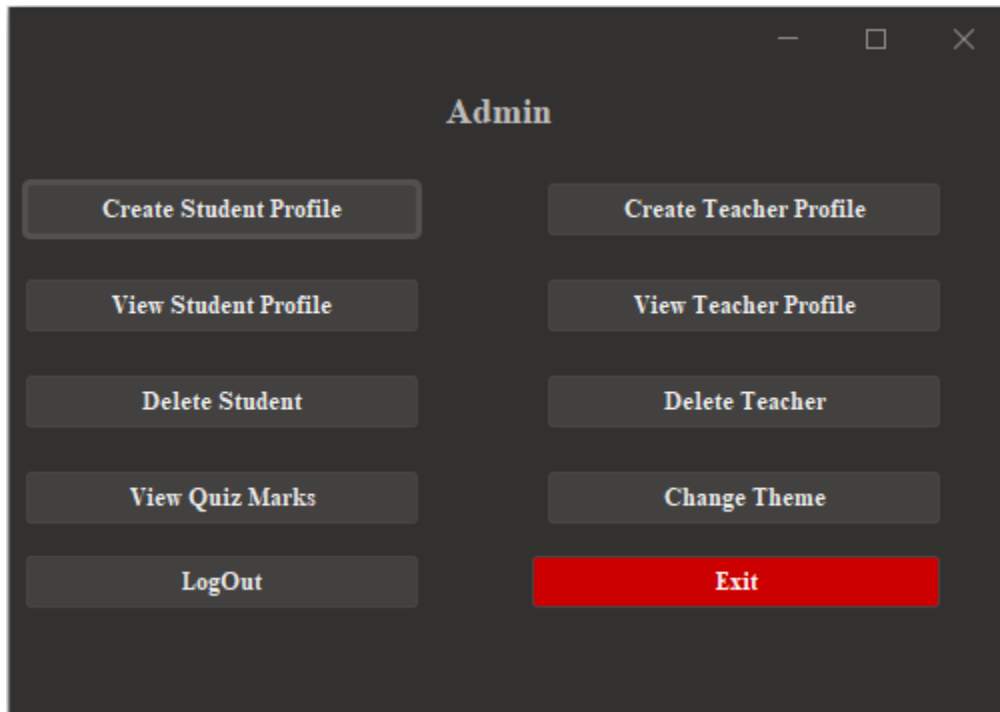
- No icons are used in the current design.

-
- 9.1.4. Imagery

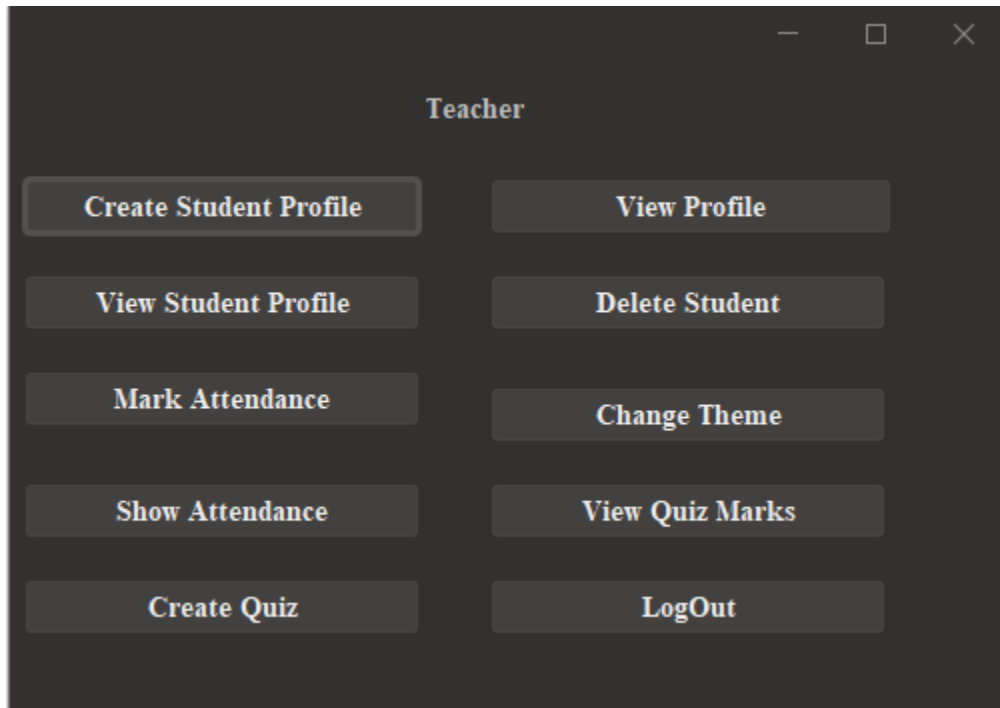
- **Log In Screen**



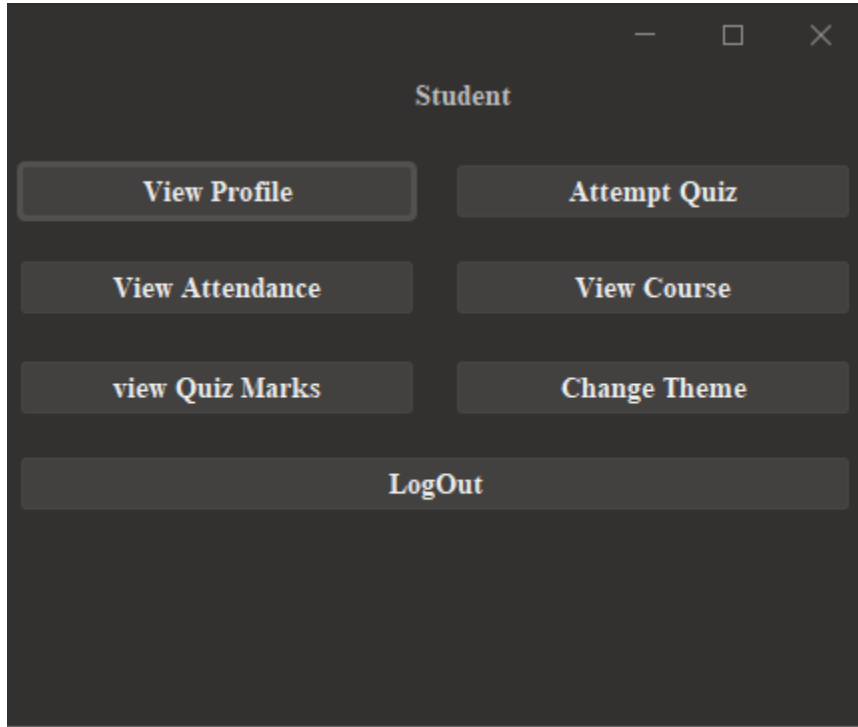
- **Admin Screen**



- **Teacher Screen**



- **Student Screen**



9.2. Mockups or Wireframes

- The provided screenshots serve as the primary mockups for the application's UI.
- These mockups represent the final visual design and interaction flow of each screen.

9.3. Explanation of Key Screens

9.3.1. Log In Screen

- **Purpose:** This is the initial entry point for all users.
- **Key Elements:**
 - Role Selection (Student, Teacher, Admin) via radio buttons.
 - ID and Password input fields.
 - "Login" button.
 - "Exit" button.
- **Interaction Flow:**
 - User selects their role.
 - User enters their ID and password.
 - Clicking "Login" validates credentials and directs the user to the appropriate dashboard (Admin, Teacher, or Student).
 - Clicking "Exit" closes the application.

9.3.2. Admin Dashboard

- **Purpose:** Provides a central hub for administrators to manage various aspects of the application.

- **Key Elements:**
 - Buttons for:
 - Creating/Viewing/Deleting Student/Teacher Profiles
 - Viewing Quiz Marks
 - Changing Theme
 - "LogOut" button.
 - "Exit" button.
- **Interaction Flow:**
 - Administrators access this dashboard after successful login.
 - Clicking on any button triggers the corresponding action or navigates the user to the relevant screen.
 - "LogOut" clears the session and redirects to the Log In screen.
 - "Exit" closes the application.

9.3.3. Teacher Dashboard

- **Purpose:** Enables teachers to manage their classroom activities and interact with student data.
- **Key Elements:**
 - Buttons for:
 - Creating/Viewing/Deleting Student Profiles
 - Marking/Showing Attendance
 - Viewing Quiz Marks
 - Creating Quizzes
 - Viewing Profile
 - Changing Theme
 - "LogOut" button.
- **Interaction Flow:**
 - Teachers access this dashboard after successful login.
 - Clicking on any button triggers the corresponding action or navigates the user to the relevant screen.
 - "LogOut" clears the session and redirects to the Log In screen.

9.3.4. Student Dashboard

- **Purpose:** Provides students with access to relevant educational resources and information.
- **Key Elements:**
 - Buttons for:
 - Viewing Profile
 - Attempting Quizzes
 - Viewing Attendance
 - Viewing Courses
 - Viewing Quiz Marks
 - Changing Theme
 - "LogOut" button.
- **Interaction Flow:**
 - Students access this dashboard after successful login.
 - Clicking on any button triggers the corresponding action or navigates the user to the relevant screen.
 - "LogOut" clears the session and redirects to the Log In screen.

-

9.4. Interaction Flow

- **General Flow:**

-

1. Log In: User selects role and enters credentials.

-

2. Authentication: System verifies credentials.

-

3. Dashboard: User is directed to the appropriate dashboard (Admin, Teacher, or Student).

-

4. Navigation: User interacts with buttons to access different functionalities.

-

5. Actions: System performs the requested action (e.g., creating a profile, viewing quiz marks).

-

6. Log Out: User logs out, clearing the session and returning to the Log In screen.

-

7. Exit: User closes the application.

- **Error Handling:**

-

- Invalid Login: Display an error message if incorrect credentials are entered.

-

- System Errors: Display appropriate error messages for unexpected system issues.

Class Diagram

